C4 Architecture

SwiftUI • SQLite • NestJS • Ports & Adapters • RAG

Память → подходящий контекст → действие

«Кстати» — это не место, куда ты складываешь информацию. Это место, откуда информация возвращается в твою жизнь.

C1 · Context

Человек, «Кстати», сервисы iOS и AI-провайдер.

C2 · Containers

iOS + локальная SQLite + NestJS AI backend.

C3 · iOS

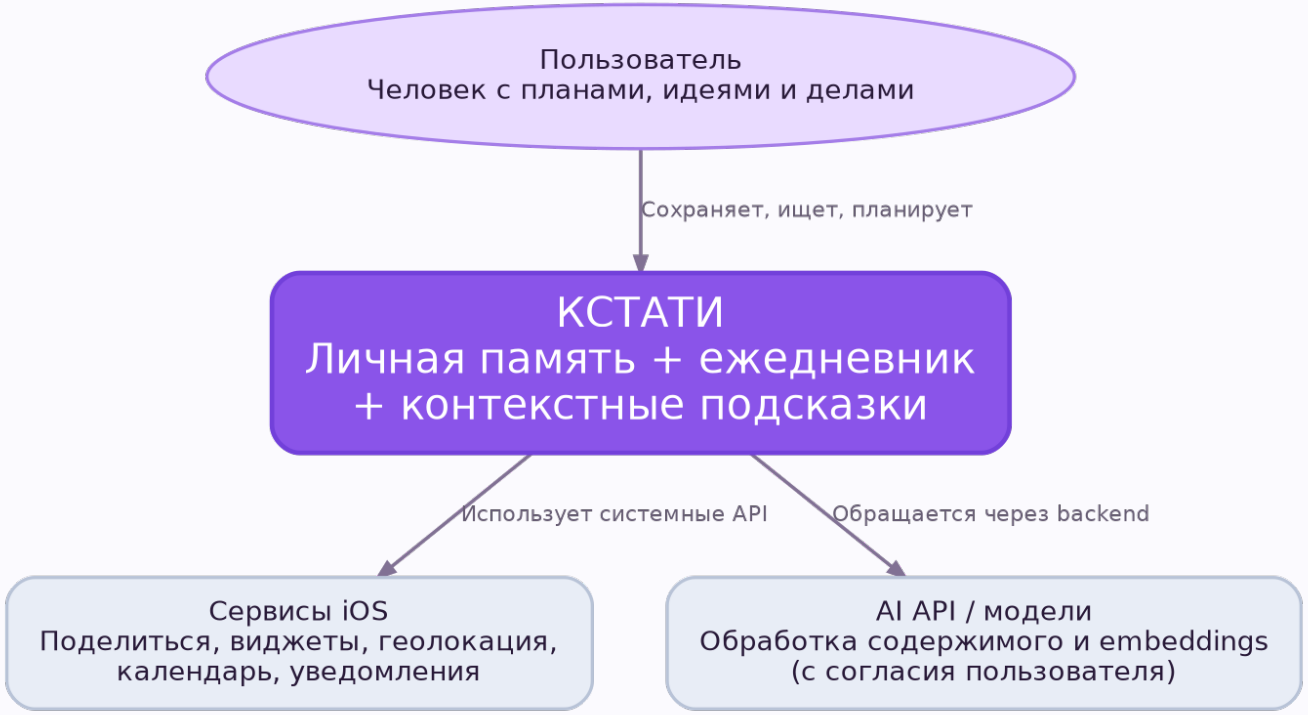
Capture, CQRS-lite, Memory, Search, Context Engine.

C3 · Backend

REST, application handlers, порты и AI-адаптеры.

C1 · Контекст системы

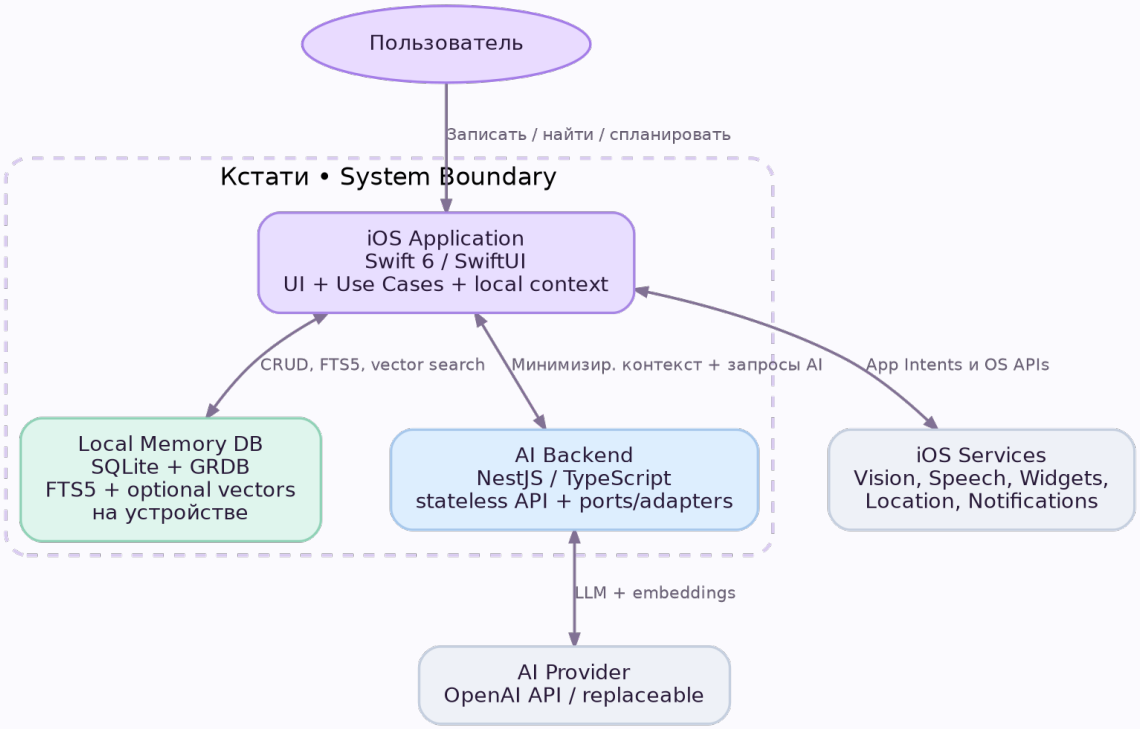
Кто взаимодействует с «Кстати» и какие внешние сервисы ей нужны



- Человек сохраняет мысли, планы, записи к врачу, рецепты и желания.
- Граница системы: «Кстати»; iOS предоставляет системные интеграции, AI Provider вызывается только через backend.
- Приватность: чувствительные сведения обрабатываются в облаке только с разрешения пользователя.

C2 · Контейнеры

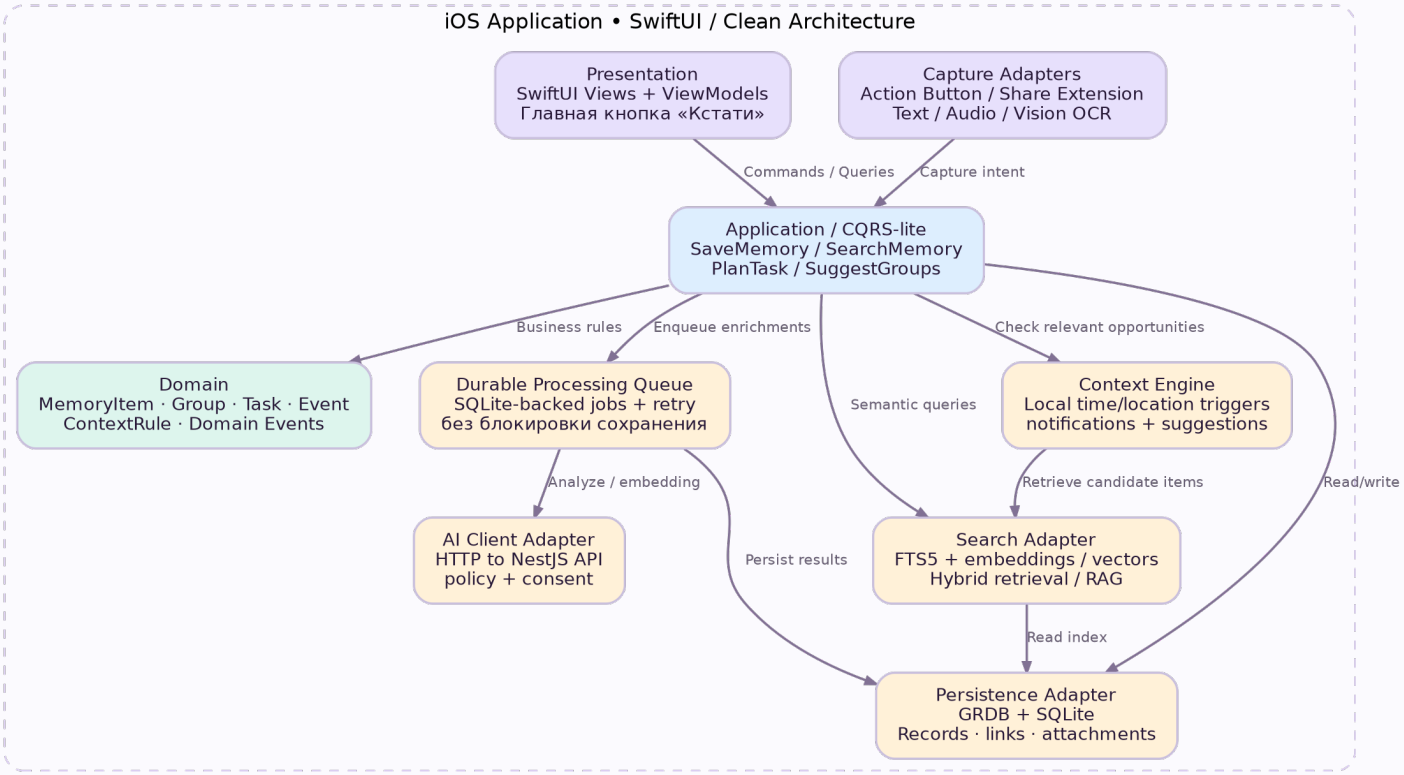
Что физически выполняется на телефоне, а что — на сервере



- iOS + SQLite: основная работа приложения и персональная память на устройстве.
- NestJS + TypeScript: API для AI-обогащения; OpenAI через заменяемые порты и адаптеры.
- Без интернета: новая запись сохраняется сразу, AI-обработка откладывается в локальную очередь.

C3 · Компоненты iOS

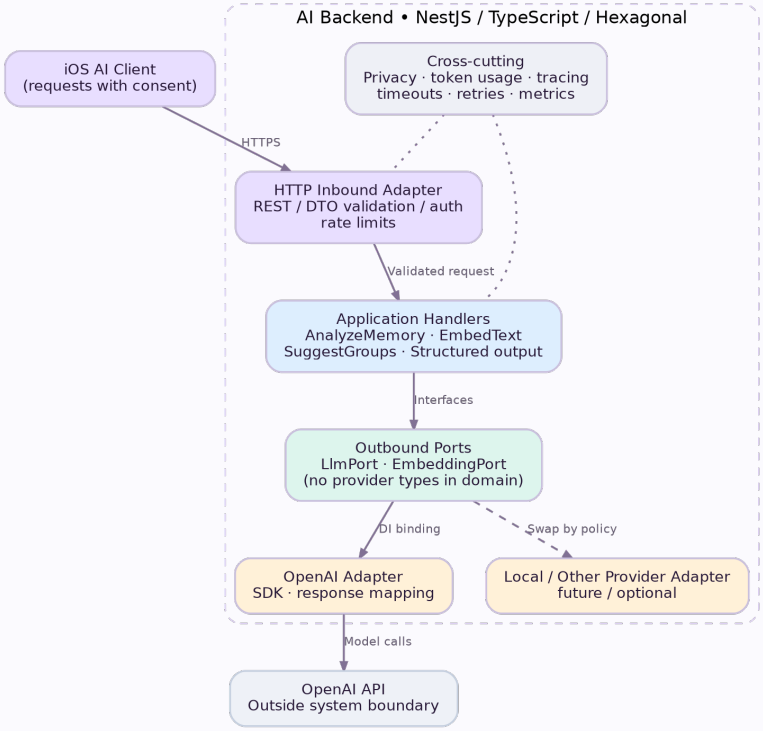
Clean Architecture, DDD-lite, CQRS-lite и локальный контекст



- Domain: MemoryItem, Group, Task, CalendarEvent; связи N:M.
- Search: FTS5 + optional embeddings; запросы к памяти выполняются локально.
- Durable jobs: повторяемая фоновая AI-обработка без блокировки сохранения.

C3 · Компоненты backend

Hexagonal Architecture: OpenAI не протекает в домен



- Inbound: OpenAPI REST, DTO validation, authentication, quotas.
- Application: AnalyzeMemory, EmbedText, SuggestGroups и schema-validated output.
- Ports: LlmPort и EmbeddingPort; OpenAIAdapter — инфраструктура. Без LangChain / LangGraph / LlamaIndex.

ADR + GoF

Что зафиксировано, а что сознательно отложено

Технологический baseline

- iOS: Swift 6 / SwiftUI / App Intents
- Storage: SQLite + GRDB + FTS5
- Backend: NestJS + TypeScript
- AI: OpenAI adapter / typed ports
- Архитектура: DDD-lite + CQRS-lite + Clean
- RAG: локальный hybrid retrieval

GoF: использовать по необходимости

- Adapter: да, provider/API/OS services
- Command: typed handlers для CQRS
- Strategy: при выборе AI или retrieval
- Observer: events + durable jobs
- Composite: позже для сложных правил
- CoR: нет, явная orchestration

Ограничения и инварианты

1. MemoryItem хранится один раз; группы не фиксированы и связываются N:M. 2. Оригинал записи нельзя перезаписывать AI-интерпретацией. 3. Никаких отдельной CQRS infrastructure, Kafka и LangGraph без доказанной необходимости. 4. API-ключи живут на сервере; облачный AI не получает данные без consent.

Как разрабатываем и проверяем

OpenSpec → реализация → тесты → GitHub Actions → TestFlight

01 · OpenSpec / SDD

proposal → specs (Given / When / Then) → design → tasks → verify → archive

02 · Unit + Integration

Swift Testing; backend Jest/Vitest; GRDB/SQLite, FTS5, domain invariants, dynamic groups

03 · iOS E2E

XCUITest: capture, offline save, search, many-to-many groups, calendar, relaunch

04 · GitHub Actions

Validation + lint + build + tests on PR; TestFlight через Fastlane по ручному approval

MVP / Первая поставка

Делаем: одна кнопка «Кстати», захват текста и скриншотов, Vision OCR, локальная память, динамические группы, базовый ежедневник, FTS5-поиск, AI-enrichment по согласию и обязательные тесты. Позже: сложные геоусловия, мультидевайс-синхронизация, более умные рекомендации и отдельная vector DB при измеренной необходимости.